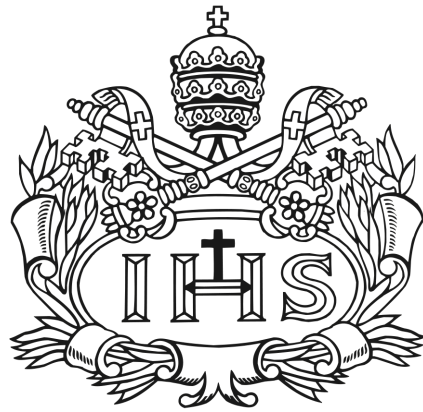


Pontificia Universidad Javeriana.



Pontificia Universidad
JAVERIANA
Colombia

Hilos - Sockets

Presentado Por: Marianne Coy y Daniel Diaz

8 de Mayo, 2026

1 Objetivos

1.1 Objetivo general

Comprender e implementar los mecanismos básicos de comunicación entre procesos distribuidos en Java mediante el uso de sockets TCP/UDP, y los mecanismos de programación concurrente mediante hilos (Thread y Runnable).

1.2 Objetivos específicos

Implementar una arquitectura cliente-servidor utilizando sockets UDP (no orientados a conexión) y sockets TCP (orientados a conexión), e identificar las diferencias prácticas entre ambos.

Comparar tres formas de resolver un mismo problema concurrente: ejecución secuencial, herencia (extends Thread) y composición (implements Runnable).

Medir y analizar empíricamente la mejora de tiempo de ejecución (speed-up) que aporta la concurrencia frente a la versión secuencial.

Entregar código membretado, documentado y un repositorio limpio, reproducible, sin ejecutables ni binarios.

2 Marco teórico y diferencias

2.1 Sockets TCP vs. UDP

Un socket es la abstracción que ofrece el sistema operativo para que dos procesos (locales o remotos) se comuniquen a través de la red. La capa de transporte ofrece dos protocolos principales:

Característica	TCP	UDP
Orientación	Orientado a conexión (handshake de 3 vías).	No orientado a conexión.
Contabilidad	Garantiza entrega y orden; retransmite si pierde paquetes.	No garantiza entrega ni orden.
Sobrecarga	Mayor (cabeceras de 20 bytes, ACKs, control de ujo).	Menor (cabeceras de 8 bytes).
Velocidad	Más lento por la lógica de control.	Más rápido y ligero.
Tipo de datos	Flujo de bytes (stream).	Datagramas independientes. API en Java Socket / ServerSocket. DatagramSocket /

Casos de uso	HTTP, SSH, bases de datos, transferencia de archivos.	DNS, streaming en vivo, VoIP, juegos en línea.
--------------	---	--

Tabla 1: Comparativa entre los protocolos TCP y UDP.

2.2 Threads en Java: extends Thread vs. implements Runnable

Java ofrece dos formas canónicas de crear hilos:

1. Heredando de Thread: la clase de negocio extiende `java.lang.Thread` y sobrescribe `run()`. La instancia es a la vez la lógica y el hilo.
2. Implementando Runnable: la clase de negocio implementa la interfaz `Runnable` y luego se entrega a un `new Thread(runnable).start()`. La lógica queda desacoplada del mecanismo de concurrencia.

Aspecto	extends Thread	implements Runnable
Herencia	Bloquea la única herencia disponible en Java.	La clase se puede heredar de otra.
Acoplamiento	Mezcla negocio y mecanismo de hilos.	Separa responsabilidades (SRP)
Reutilización	La clase es un hilo concreto.	El mismo Runnable puede ejecutarse en varios hilos o en un <code>ExecutorService</code>
Recomendación	Solo en casos muy simples.	Recomendado en general

Tabla 2: Comparativa entre los dos enfoques de creación de hilos en Java.

3 Funcionamiento de los programas

3.1 Parte 1: Sockets

Se implementan cuatro programas:

- `serUDPsocket.java` y `cliUDPsocket.java` servidor y cliente UDP en el puerto 6000. El cliente envía mensajes de texto leídos por teclado y termina cuando el usuario escribe fin. El servidor imprime cada datagrama recibido y termina al recibir fin.
- `serTCPsocket.java` y `cliTCPsocket.java` servidor y cliente TCP en el puerto 6001. La conexión se establece con `Socket/ServerSocket` y los mensajes se transmiten con `writeUTF/readUTF` sobre flujos de datos.

Figura 1: Comunicación UDP: envío de datagramas independientes sin conexión

Figura 2: Comunicación TCP: handshake de tres vías antes del envío de datos.

3.2 Parte 2: Threads

Se modela un supermercado con dos cajeras y dos clientes. Cada cliente trae un carro de compra representado como un arreglo de enteros, donde cada entero es el tiempo (en segundos) que tarda el procesamiento de un producto.

- Cliente 1: [2, 2, 1, 5, 2, 3] → 15 segundos.
- Cliente 2: [1, 3, 5, 1, 1] → 11 segundos.

Se implementan tres versiones:

- Main.java: las cajeras se ejecutan secuencialmente. La segunda cajera no comienza hasta que la primera termina.
- MainThread.java: cada cajera es una instancia de CajeraThread, que hereda de Thread. Se invoca start() en cada una.
- MainRunnable.java: cada cajera es un Cajera (POJO) envuelto en un Runnable ejecutado por un Thread independiente.

4. Experimentos y pruebas

4.1 Pruebas de sockets UDP

Se inicia el servidor con make ser-udp y luego el cliente con make cli-udp HOST=localhost. Desde el cliente se envían mensajes por teclado, y el servidor los imprime.

```
estudiante@MIG129:~/Hilos-Sockets$ make run-main
=== Ejecutando Main (SECUENCIAL) ===
java -cp build/threads tallerThreads.Main
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 15seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 16seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 19seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 24seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 25seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 26seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 26seg
estudiante@MIG129:~/Hilos-Sockets$ make ser-udp
java -cp build/sockets serUDPSocket
Prueba de sockets UDP (servidor)
Creando socket... ok
Recibiendo mensajes...
hola

```

Figura 3: Servidor UDP recibiendo mensajes desde el cliente.

```
estudiante@MIG114:~/Hilos-Sockets$ make cli-udp HOST=10.43.99.70
javac -d build/sockets TCP_UDP/cliTCPsocket.java TCP_UDP/cliUDPsocket.java TCP_UDP/serTCPsocket.java TCP_UDP/serUDPsocket.java
java -cp build/sockets cliUDPsocket 10.43.99.70
Prueba de sockets UDP (cliente)
Creando socket... ok
Capturando direccion de host... ok
Introduce mensajes a enviar:
hola

```

Figura 4: Cliente UDP enviando mensajes al servidor.

4.2 Pruebas de sockets TCP

Análogo al caso UDP, pero con make ser-tcp y make cli-tcp HOST=localhost.

```

estudiante@MIG129:~/Hilos-Sockets$ make ser-tcp
java -cp build/sockets serTCPsocket

==**SOCKETS TCP <<SERVIDOR>>
Colombia Campeona del mundo 2026
James el mejor jugador
Fin

```

Figura 5: Servidor TCP recibiendo mensajes desde el cliente.

```

estudiante@MIG114:~/Hilos-Sockets$ make cli-tcp HOST=10.43.99.70
java -cp build/sockets cliTCPsocket 10.43.99.70
Prueba de sockets TCP (CLIENTE)
Capturando direccion de host... ok
Creando socket... ok
Introduce mensajes a enviar:
Colombia Campeona del mundo 2026
James el mejor jugador
Fin

```

Figura 6: Cliente TCP enviando mensajes al servidor.

4.3 Pruebas de threads

Se ejecutaron las tres versiones desde la misma máquina y se midieron los tiempos totales mediante las trazas que imprime el propio programa (timestamp relativo al instante inicial).

```

estudiante@MIG129:~/Hilos-Sockets$ make run-main
=== Ejecutando Main (SECUENCIAL) ===
java -cp build/threads tallerThreads.Main
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 15seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 16seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 19seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 24seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 25seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 26seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 26seg
estudiante@MIG129:~/Hilos-Sockets$

```

Figura 7: Salida de la ejecución secuencial (make run-main).

```

estudiante@MIG129:~/Hilos-Sockets$ make run-thread
=== Ejecutando MainThread (extends Thread) ===
java -cp build/threads tallerThreads.MainThread
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 0seg
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
estudiante@MIG129:~/Hilos-Sockets$

```

4.4 Comparativa de tiempos

```

estudiante@MIG129:~/Hilos-Sockets$ make run-thread
=== Ejecutando MainThread (extends Thread) ===
java -cp build/threads tallerThreads.MainThread
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 0seg
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
estudiante@MIG129:~/Hilos-Sockets$

```

Figura 8: Salida de MainThread (extends Thread).

```

estudiante@MIG129:~/Hilos-Sockets$ make run-runnable
=== Ejecutando MainRunnable (implements Runnable) ===
java -cp build/threads tallerThreads.MainRunnable
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
estudiante@MIG129:~/Hilos-Sockets$

```

Figura 9: Salida de MainRunnable (implements Runnable).

5. Observaciones

- Imports faltantes en el código original. Los archivos cliTCPsocket.java y serTCPsocket.java originales no incluían los imports de java.net.* y java.io.* se agregaron en la versión entregada.
- Bug sutil en cliente UDP. El código original construía el DatagramPacket con mensaje.length()(número de caracteres) en lugar de mensaje_bytes.length (número de bytes).

Esto puede truncar mensajes con caracteres multi-byte (acentos, ñ, emojis). Se corrigió en la versión entregada.

- Bucle infinito en servidor TCP. El código original usa `while (1>0)`; en producción se debería controlar la terminación por `EOFException` (cierre del cliente) o por un mensaje convencional como `fin`.
- No determinismo en el orden de los hilos. En las versiones concurrentes, el orden de las trazas varía entre ejecuciones. Esto es esperable: el scheduler del sistema operativo decide qué hilo ejecuta primero.

6. Conclusiones

- TCP y UDP responden a casos de uso fundamentalmente distintos. TCP es la opción adecuada cuando se requiere confiabilidad, orden garantizado y control de flujo, a costa de mayor sobrecarga (cabeceras de 20 bytes, ACKs, retransmisiones). UDP es preferible cuando se prioriza velocidad y bajo overhead (cabeceras de 8 bytes, sin establecimiento de conexión), asumiendo que la aplicación tolera pérdidas o las maneja a nivel superior. Las pruebas realizadas evidenciaron esta diferencia: TCP requiere que el servidor esté escuchando antes de cualquier intento de conexión, mientras que UDP permite "disparar y olvidar" sin retroalimentación de éxito.
- `Runnable` es preferible a `Thread` en general. Aunque ambas alternativas alcanzan exactamente el mismo rendimiento en términos de tiempo de ejecución, el enfoque basado en `Runnable` no consume la única herencia disponible en Java (al ser una interfaz y no una clase) y desacopla la lógica de negocio del mecanismo de ejecución concurrente. Esto facilita su uso con frameworks modernos como `ExecutorService` y `thread pools`, y respeta el principio de responsabilidad única (SRP). En código de producción, `Runnable` o sus evoluciones (`Callable`, `CompletableFuture`) son la recomendación estándar.
- La depuración de programas concurrentes requiere atención adicional. El no determinismo en el orden de las trazas y la posibilidad latente de condiciones de carrera obligan a diseñar las pruebas de forma robusta frente a esta variabilidad. En este taller no hubo `race conditions` porque las cajas procesan clientes independientes (sin recursos compartidos), pero en problemas más complejos esto requeriría mecanismos de sincronización como `synchronized`, `Lock`, o estructuras de datos concurrentes (`ConcurrentHashMap`, etc.).
- Los sockets en Java abstraen exitosamente los detalles de bajo nivel del protocolo. Las clases `Socket/ServerSocket` (TCP) y `DatagramSocket/DatagramPacket` (UDP) ocultan la complejidad del protocolo subyacente (handshakes, segmentación, números de secuencia) y permiten al programador enfocarse en la lógica de comunicación. Esto se evidenció en lo simple que resultó implementar un sistema cliente-servidor distribuido entre dos máquinas físicas distintas con apenas 30 líneas de código por archivo.
- La comunicación distribuida entre máquinas físicas distintas funciona correctamente con configuración mínima. Las pruebas se realizaron entre dos servidores Linux de la Javeriana en la misma red institucional, sin necesidad de configurar firewalls explícitamente ni túneles especiales. La diferencia entre comunicación `localhost` y comunicación entre máquinas reales es transparente al programador desde el punto de vista del código: la única variable que cambia es la dirección IP del servidor que recibe el cliente.

- La concurrencia bien aplicada produce un speed-up medible. En este taller se observó speed-up una reducción de 26 s a 15 s (1.73x) al pasar de ejecución secuencial a ejecución con dos hilos.

Versión	Mecanismo	Tiempo total	Speed-up
Main	Secuencial	26 s	1.00x
MainThread	extends Thread	15 s	1.73x
MainRunnable	implements Runnable	15 s	1.73x

Tabla 3: Tiempos de ejecución observados en las tres versiones.